

AMAZON FRESH ANALYTICS

Project Documentation

A Relational Database & SQL Analytics Project

Domain: E-commerce & Retail Analytics
Database: MySQL

Prepared by: DHARNISH K
Vinsup Skill Academy, Coimbatore

1. Introduction

Amazon Fresh is an e-commerce platform specializing in groceries and daily essentials. As the platform grows, management needs to optimize operations, improve customer satisfaction, and increase revenue. Doing this requires effectively organizing and analyzing large volumes of data spanning customers, products, suppliers, orders, and reviews.

This project designs a relational database for Amazon Fresh and uses SQL to answer real business questions — from identifying top-performing products to understanding customer spending patterns and keeping inventory aligned with demand.

2. Objectives

- Design a normalized relational schema that accurately models Amazon Fresh's business entities and their relationships.
- Apply DDL and constraints to enforce data integrity (primary keys, NOT NULL, CHECK, DEFAULT, UNIQUE).
- Use DML operations to insert, update, and delete records safely.
- Write SQL queries — filtering, aggregation, joins, subqueries, and window functions — to answer business questions.
- Normalize the Products table to Third Normal Form (3NF) by separating categories and subcategories.
- Translate query results into actionable business insights for stakeholders.

3. Tools & Technologies

Component	Details
Database Engine	MySQL
Language	SQL (DDL, DML, DQL, TCL concepts)
Dataset	6 CSV files — Customers, Products, Suppliers, Orders, Order_Details, Reviews
Techniques Used	Joins, GROUP BY / HAVING, subqueries, window functions (DENSE_RANK), normalization

4. Database Schema

The database ("amazon") consists of six core tables plus two normalization tables created during Task 12.

4.1 Core Tables

Table	Key Columns	Purpose
Customers	CustomerID (PK), Name, City, Age, PrimeMember	Customer demographics and Prime membership status
Products	ProductID (PK), ProductName, Category, SubCategory, PricePerUnit, StockQuantity, SupplierID (FK)	Product catalog, pricing, and stock levels
Suppliers	SupplierID (PK), Name, City, Contact	Supplier details and locations
Orders	OrderID (PK), CustomerID (FK), OrderDate, OrderAmount, DeliveryFee, DiscountApplied	High-level order records
Order_Details	OrderDetailID (PK), OrderID (FK), ProductID (FK), Quantity, UnitPrice, Discount	Line items within each order

Table	Key Columns	Purpose
Reviews	ReviewID (PK), ProductID (FK), CustomerID (FK), Rating (1–5), ReviewText	Customer feedback and ratings per product

4.2 Normalization Tables (Task 12 — 3NF)

To remove redundancy from the Products table, category data was split out into two dedicated tables:

- productCategory — maps each product to a Category and an associated attribute (color code) via a surrogate vera_id.
- SubCategory — maps each product to its SubCategory, linked back to productCategory via vera_id.

This separates repeating category/subcategory text out of Products, so category information is stored once and referenced by key — satisfying 3NF.

4.3 Relationships

- Customers (1) → Orders (many): one customer can place many orders.
- Orders (1) → Order_Details (many): one order can contain many line items.
- Products (1) → Order_Details (many): one product can appear in many order line items.
- Suppliers (1) → Products (many): one supplier can supply many products.
- Customers (1) → Reviews (many) and Products (1) → Reviews (many): customers review products.
- productCategory (1) → SubCategory (many): one category can have multiple subcategories.

5. Approach / Methodology

5.1 Data Modeling & Basic Queries

The schema was mapped out first (entities, keys, relationships), followed by simple lookups such as retrieving customers by city and filtering products by category to validate the schema against real query patterns.

5.2 DDL & Constraints

The Customers table was constrained with a primary key on CustomerID and an Age check. A DEFAULT constraint sets PrimeMember to "No" when not specified, and a CHECK constraint on Reviews restricts Rating to the 1–5 range.

5.3 DML Operations

New product rows were inserted via INSERT statements, an existing product's stock was adjusted with UPDATE, and a supplier record was removed with DELETE (after disabling MySQL's safe-update mode for the operation).

5.4 Aggregation & Business Queries

GROUP BY and HAVING were used to summarize data at the product and customer level (e.g., average rating per product, total sales per product). Multi-table JOINS combined Orders, Order_Details, Customers, and Products to compute spending and revenue metrics. DENSE_RANK() was applied to rank customers by total spending.

5.5 Normalization

The Products table's category and subcategory columns were extracted into productCategory and SubCategory tables, linked by key, to reduce redundancy and move the design toward 3NF.

5.6 Subqueries & Business Insights

Subquery-style joins with GROUP BY / ORDER BY / LIMIT identified the top 3 products by revenue. Aggregate queries over Customers and Products surfaced which cities have the most Prime members and which categories are ordered most often.

6. Key Business Questions Answered

#	Business Question	Technique Used
1	Which customers are in a given city?	WHERE filter
2	Which products fall under a given category?	WHERE filter
3	What is each customer's total spending?	JOIN + SUM aggregation
4	How do customers rank by spending?	JOIN + DENSE_RANK() window function
5	Which customers spent over ₹5,000?	JOIN + GROUP BY + HAVING
6	What is the total revenue per order?	JOIN between Orders and Order_Details + SUM
7	Which customers ordered most in a given period?	JOIN + WHERE + COUNT + GROUP BY
8	Which supplier holds the most stock?	GROUP BY + SUM + ORDER BY
9	What are the top 3 products by revenue?	JOIN + GROUP BY + ORDER BY + LIMIT
10	Which cities have the most Prime members?	WHERE + GROUP BY + COUNT
11	What are the top 3 most-ordered categories?	GROUP BY + COUNT + ORDER BY + LIMIT

7. Assumptions

- Each CSV file maps directly to one table with consistent column types as specified in the dataset description (INT, VARCHAR, DECIMAL, DATE, TINYINT, TEXT).
- "Total spending" per customer is computed as order amount plus delivery fee, minus any discount applied.
- "Total sales" per product is computed as quantity multiplied by unit price, minus discount, aggregated across all of that product's order lines.
- A "specific time period" for order-frequency analysis is represented using a single OrderDate filter as a stand-in for a date-range filter.
- PrimeMember is stored as a Yes/No text value rather than a strict boolean, matching how the source CSV represents it.

8. Findings & Business Insights

Running the queries in Section 6 against the loaded dataset produces directly actionable output for each use case defined in the problem statement:

- Customer Insights — the customer-spending and ranking queries surface the highest-value customers and the cities where they're concentrated, supporting targeted marketing.
- Product Management — category- and stock-level aggregations show which products and categories are moving fastest, informing restocking priorities.
- Supplier Analysis — the stock-by-supplier query identifies which suppliers are carrying the most inventory, useful for supply-chain planning.

- Revenue Analysis — the per-order and per-product revenue queries highlight which products and orders drive the most income.
- Customer Satisfaction — the average-rating query flags products with strong (or weak) review performance for merchandising decisions.

Note: exact figures (top city, top product, etc.) depend on the data actually loaded into the tables; the queries in the SQL script are written to surface them directly once run against the live dataset.

9. Deliverables

- SQL Scripts — `sqlprojectmini.sql` (schema setup, DDL, DML, constraints, and all business queries).
- ER Diagram — visual representation of table relationships (Customers, Products, Suppliers, Orders, Order_Details, Reviews, and the normalized category tables).
- Reports — query outputs addressing each business question in Section 6.
- Documentation — this document, covering approach, assumptions, and findings.

10. Conclusion

This project delivers a working relational database for Amazon Fresh together with a set of SQL queries that directly answer the platform's key operational and business questions. The schema enforces data integrity through constraints, reduces redundancy through normalization, and the query layer turns raw transactional data into insights that support marketing, inventory, supplier, and revenue decisions.